

Penerapan Directed Acyclic Graph Terhadap Jalur KRL Jabodetabek

- Tugas Besar Mata Kuliah Struktur Data (SD2104) -

1st Araf Rahman Maulana
122450002

2nd Meira Listyaningrum
1224500011

3rd Febiya Jomy Pratiwi
122450074

4th Farrel Julio Akbar
122450110

5th Amalia Melani Putri
122450122

Abstrak

Penelitian ini merekomendasikan penggunaan Directed Acyclic Graph (DAG) untuk memodelkan KRL di wilayah Jabodetabek. Diharapkan bahwa DAG akan memberikan pemodelan terbaik, menjamin operasional yang lancar, dan meningkatkan kepuasan pengguna KRL di wilayah tersebut.

DAG digunakan untuk menunjukkan jalur perjalanan KRL Jabodetabek adalah masalah utama penelitian. Proses representasi stasiun dan jalur perjalanan dengan DAG sangat penting untuk penggunaan teknologi ini.

Data yang digunakan berasal dari dataset peta rute KRL Jabodetabek yang diperoleh dari situs Commuterline Indonesia. Dataset diolah kemudian akan dilakukan analisis pemodelan jalur KRL.

Dalam desain sistem proyek ini, bahasa pemrograman Python digunakan

bersama dengan library Networkx dan Matplotlib. Library pertama digunakan untuk memanipulasi dan mempelajari struktur jaringan kompleks, dan library kedua digunakan untuk membuat visualisasi data yang informatif.

Kata Kunci: Dataset, KRL, library, python.

I. Pendahuluan

A. Latar Belakang

Kota Jakarta, sebagai ibu kota negara dengan banyak aktivitas, tentunya memiliki banyak orang yang terlibat dalam sistem yang berlaku di dalamnya. Sebagian besar orang pergi ke luar Kota Jakarta karena banyaknya aktivitas, layanan dan fasilitas yang ada di sana. Mereka berusaha mengadu nasib untuk mendapatkan hasil yang lebih baik dari apa yang mereka dapatkan di tempat asal mereka. Dengan itu mereka memerlukan mobilitas transportasi

Salah satu cara untuk memenuhi kebutuhan manusia, terutama dalam hal perpindahan manusia atau barang dari satu tempat ke tempat lainnya, adalah transportasi. Namun, kemacetan sangat mungkin karena banyaknya transportasi kendaraan pribadi di Indonesia. Oleh karena itu, tidak jarang orang lebih memilih menggunakan kereta listrik(KRL). Ini karena mereka memiliki kapasitas yang cukup besar untuk orang dan barang, harga tiket yang murah, dan menghindari macet [1].

Untuk mempertahankan loyalitas pelanggan dan meningkatkan kualitas layanan, penting untuk meningkatkan pelayanan KRL agar menjadi opsi transportasi utama di Jabodetabek[2].

Penelitian ini akan mengusulkan sebuah pemodelan KRL untuk wilayah Jabodetabek, yaitu Jakarta, Bogor, Depok, Tangerang, dan Bekasi. Untuk menggambarkan jalur perjalanan KRL Jabodetabek, pemodelan menggunakan Graph Acyclic Directed (DAG). Metode ini diharapkan dapat menghasilkan pemodelan terbaik, menjamin keberlangsungan operasional, dan memberikan kepuasan terbaik kepada pengguna KRL di Jabodetabek.

B. Rumusan Masalah

Bagian penting dari penggunaan Directed Acyclic Graph (DAG) terhadap jalur KRL Jabodetabek, yaitu cara menggunakan Directed Acyclic Graph (DAG) untuk merepresentasikan jalur perjalanan. Proses dan teknik untuk

merepresentasikan stasiun-stasiun dan jalur perjalanan KRL Jabodetabek dengan DAG sangat penting untuk pemahaman dan penggunaan teknologi ini.

Pertanyaan penting yang harus diselesaikan adalah bagaimana stasiun digambarkan sebagai simpul dalam graf dan bagaimana tepi arah antara stasiun mencerminkan jalur perjalanan khusus antar-stasiun.

C. Deskripsi Data

Pada kasus ini digunakan dataset yang diperoleh dari situs resmi *commuterline indonesia* (<https://commuterline.id/perjalanan-krl/peta-rute>) berupa data peta rute .

Data tersebut akan diolah sedemikian rupa untuk menghasilkan analisis pemodelan KRL untuk wilayah Jabodetabek

II. Rancangan Sistem

A. Gambaran Umum

Proyek ini membahas tentang Directed Acyclic Graph (DAG) terhadap jalur KRL Jabodetabek menggunakan program pemrograman *Python* dengan *library Networkx* dan *Matplotlib*.

DAG adalah grafik berarah yang tidak memiliki *loop* di dalamnya [3]. *Library Networkx* digunakan untuk mempelajari grafik dan jaringan, dan

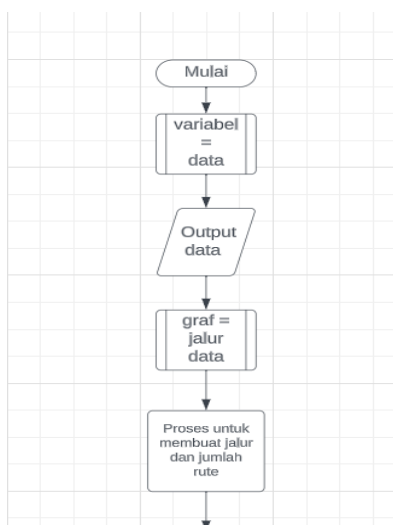
Matplotlib merupakan *library* yang digunakan untuk membuat grafik garis, scatter plot, histogram, dan banyak lainnya dari data yang dimiliki.

B. Library yang Digunakan

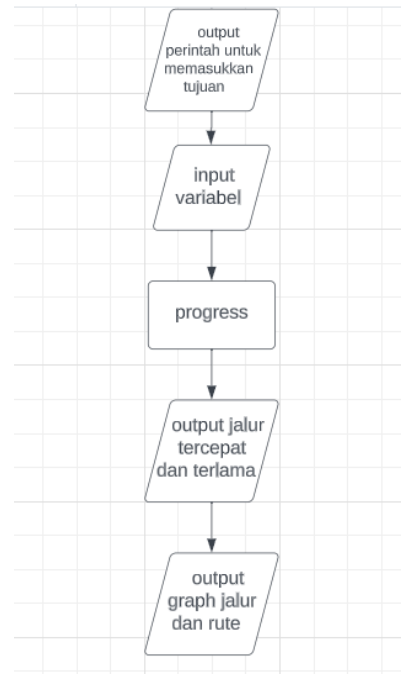
Library yang digunakan dalam proyek ini adalah:

- *Library* *Networkx*
Networkx adalah paket perangkat lunak bahasa Python untuk manipulasi, penciptaan, dan studi tentang struktur, dinamika, dan fungsi dari jaringan yang kompleks [4].
- *Library* *Matplotlib*
Matplotlib adalah *library* yang digunakan dalam bahasa pemrograman Python untuk membuat visualisasi data yang menarik dan informatif dengan grafik[5].

C. Flowchart



Gambar 2.1



Gambar 2.2

D. Daftar Fungsionalitas

1. Import network as nx
2. Import matplotlib.pyplot as plt
3. Directed
4. Start
5. End
6. Calculate total distance
7. Graph
8. Path

III. Hasil dan Pembahasan

Pemrograman ini menggunakan pustaka NetworkX dan Matplotlib untuk memodelkan serta memvisualisasikan Directed Acyclic Graph (DAG) yang merepresentasikan rute kereta api antar

stasiun dengan jarak antar stasiunnya sebagai bobot.

A. Import Library

```
import networkx as nx
import matplotlib.pyplot as plt
```

Gambar 3.1

Dua baris code ini digunakan untuk mengimport library yang diperlukan untuk pembuatan graf dan membuat visualisasinya. Dengan dua pustaka ini, program dapat mendefinisikan, menganalisis, dan memvisualisasikan graf berarah tanpa siklus atau Directed Acyclic Graph (DAG).

B. Fungsi-fungsi dan graf

```
def directed(graph, start, end):
    shortest_path = nx.shortest_path(graph, start, end, weight='weight')
    return shortest_path

def calculate_total_distance(graph, path):
    total_distance = sum([graph[path[i]][path[i+1]]['weight'] for i in range(len(path)-1)])
    return total_distance
```

Gambar 3.2

Def directed adalah pendefinisian fungsi yang digunakan untuk mencari jalur terpendek antar stasiun pada DAG. Di dalamnya terdapat 3 parameter, parameter graph digunakan untuk mempresentasikan rute kereta api, parameter start dan end adalah bagian stasiun awal dan akhir yang jalurnya ingin dicari.

Def caculate_total_distance adalah pendefinisian fungsi yang digunakan untuk menghitung total jarak dari suatu jalur. Di dalamnya terdapat 2 parameter, parameter pertama adalah graph yang digunakan untuk merepresentasikan rute kereta api, lalu

parameter path adalah jalur yang ingin dihitung total jaraknya. Fungsi ini menggunakan list comprehension untuk menjumlahkan bobot dari setiap edge dari jalur yang diberikan.

C. Input Data

```
def main():
    # Contoh DAG rute kereta api dengan jarak antar stasiun (dalam satuan kilometer)
    G = nx.DiGraph()

    # Menambahkan edge (rute kereta) beserta bobotnya (jarak antar stasiun)
    edges = [
        ('Bogor', 'Cilebut', {'weight': 7.518}),
        ('Cilebut', 'Bojong Gede', {'weight': 4.331}),
        ('Bojong Gede', 'Citayam', {'weight': 5.197}),
        ('Hamo', 'Cibinong', {'weight': 4.3}),
        ('Cibinong', 'Citayam', {'weight': 3.805}),
        ('Citayam', 'Depok', {'weight': 5.084}),
        ('Depok', 'Depok Baru', {'weight': 1.741}),
        ('Depok Baru', 'Pondok Cina', {'weight': 2.570}),
        ('Pondok Cina', 'Universitas Indonesia', {'weight': 1.109}),
        ('Universitas Indonesia', 'Universitas Pancasila', {'weight': 2.264}),
        ('Universitas Pancasila', 'Lenteng Agung', {'weight': 1.029}),
        ('Lenteng Agung', 'Tanjung Barat', {'weight': 2.460}),
        ('Tanjung Barat', 'Pasar Minggu', {'weight': 3.037}),
        ('Pasar Minggu', 'Pasar Minggu Baru', {'weight': 1.695}),
        ('Pasar Minggu Baru', 'Duren Kalibata', {'weight': 1.509}),
        ('Duren Kalibata', 'Cawang', {'weight': 1.475}),
        ('Cawang', 'Tebet', {'weight': 1.301}),
        ('Tebet', 'Manggarai', {'weight': 2.610}),
        ('Manggarai', 'Cikini', {'weight': 1.606}),
        ('Cikini', 'Gondangdia', {'weight': 1.699}),
        ('Gondangdia', 'Juanda', {'weight': 2.198}),
        ('Juanda', 'Sawah Besar', {'weight': 0.207}),
        ('Sawah Besar', 'Mangga Besar', {'weight': 1.717}),
        ('Mangga Besar', 'Jayakarta', {'weight': 1.020}),
        ('Jayakarta', 'Jakarta Kota', {'weight': 1.487}),
        ('Jakarta Kota', 'Kampung Bandan', {'weight': 1.364}),
        ('Kampung Bandan', 'Ancol', {'weight': 2.185}),
        ('Ancol', 'Tanjung Priok', {'weight': 4.556}),
        ('Manggarai', 'Sudirman', {'weight': 3.186}),
```

Gambar 3.3

```
('Sudirman', 'BNI City', {'weight': 0.28}),
('BNI City', 'Karet', {'weight': 0.75}),
('Karet', 'Tanah Abang', {'weight': 2.029}),
('Tanah Abang', 'Palmerah', {'weight': 3.191}),
('Palmerah', 'Kebayoran', {'weight': 3.737}),
('Kebayoran', 'Pondok Ranji', {'weight': 6.218}),
('Pondok Ranji', 'Jurang Mangu', {'weight': 2.179}),
('Jurang Mangu', 'Sudirman', {'weight': 1.974}),
('Sudirman', 'Rawa Buntu', {'weight': 4.566}),
('Rawa Buntu', 'Serpong', {'weight': 2.413}),
('Serpong', 'Cisauk', {'weight': 1.784}),
('Cisauk', 'Cicayur', {'weight': 2.519}),
('Cicayur', 'Parung Panjang', {'weight': 5.968}),
('Parung Panjang', 'Cilejit', {'weight': 7.025}),
('Cilejit', 'Daru', {'weight': 2.675}),
('Daru', 'Tenjo', {'weight': 3.902}),
('Tenjo', 'Tigaraksa', {'weight': 2.974}),
('Tigaraksa', 'Cikoya', {'weight': 2.651}),
('Cikoya', 'Maja', {'weight': 1.853}),
('Maja', 'Citeras', {'weight': 7.293}),
('Citeras', 'Rangkasbitung', {'weight': 9.847}),
('Tanah Abang', 'Duri', {'weight': 3.632}),
('Duri', 'Grogol', {'weight': 2.7}),
('Grogol', 'Pesing', {'weight': 2.036}),
('Pesing', 'Taman Kota', {'weight': 1.514}),
('Taman Kota', 'Bojong Indah', {'weight': 2.434}),
('Bojong Indah', 'Rawa Buaya', {'weight': 1.152}),
('Rawa Buaya', 'Kali Deres', {'weight': 2.584}),
('Kali Deres', 'Puris', {'weight': 2.548}),
('Puris', 'Batu Ceper', {'weight': 1.8}),
('Batu Ceper', 'Tanah Tinggi', {'weight': 2}),
('Tanah Tinggi', 'Tangerang', {'weight': 1.609}),
('Duri', 'Angke', {'weight': 1.230}),
('Angke', 'Kampung Bandan', {'weight': 4.102}),
('Kampung Bandan', 'Hajawali', {'weight': 1.444}),
```

Gambar 3.4

```

('Kampung Bandan', 'Rajawali', {'weight': 1.444}),
('Rajawali', 'Kemayoran', {'weight': 1.901}),
('Kemayoran', 'Pasar Senen', {'weight': 1.436}),
('Pasar Senen', 'Gang Sentiong', {'weight': 1.567}),
('Gang Sentiong', 'Kramat', {'weight': 0.973}),
('Kramat', 'Pondok Jati', {'weight': 1.829}),
('Pondok Jati', 'Jatinegara', {'weight': 1.236}),
('Manggarai', 'Matraman', {'weight': 1.6}),
('Matraman', 'Jatinegara', {'weight': 0.45}),
('Jatinegara', 'Klender', {'weight': 3.395}),
('Klender', 'Buaran', {'weight': 3.1}),
('Buaran', 'Klender Baru', {'weight': 1.305}),
('Klender Baru', 'Cakung', {'weight': 1.385}),
('Cakung', 'Kranji', {'weight': 3.097}),
('Kranji', 'Bekasi', {'weight': 2.520}),
('Bekasi', 'Bekasi Timur', {'weight': 2.9}),
('Bekasi Timur', 'Tambun', {'weight': 4.6}),
('Tambun', 'Cibitung', {'weight': 3.9}),
('Cibitung', 'Metland Telaga Murni', {'weight': 4.4}),
('Metland Telaga Murni', 'Cikarang', {'weight': 8.7}),
]

G.add_edges_from(edges)

```

Gambar 3.5

Def *main()*, merupakan fungsi utama dari program. Di dalamnya dibuat objek *G*, ini merupakan DAG rute kereta api menggunakan pustaka *NetworkX*. Setiap edge (rute kereta) memiliki atribut 'weight', yang menyatakan jarak antar stasiun dalam satuan kilometer.

nx.DiGraph(), digunakan untuk membuat objek graf baru. Lalu edge-edge dari data akan ditambahkan ke dalam graf 'G' menggunakan metode *add_edges_from*. Rincian-rincian dari rute kereta api beserta jaraknya akan ditentukan dalam list *edges*. Setiap elemen dari list merupakan tupel yang berisi stasiun awal, stasiun tujuan, dan bobot(jarak)--.

D. Meminta input pengguna dan visualisasi graf rute kereta api

```

G.add_edges_from(edges)

# Memasukkan stasiun awal dan stasiun tujuan akhir
start_station = input("Masukkan stasiun awal: ")
end_station = input("Masukkan stasiun tujuan akhir: ")

# Menampilkan graf setelah input stasiun
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=700, node_color='skyblue', font_size=10, font_color='black', arrows=True, font_weight='bold')
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.show()

```

Gambar 3.6

Pada bagian ini, program akan meminta inputan mengenai stasiun awal dan tujuan akhir dari pengguna. Setelah pengguna memasukkan stasiun awal dan tujuan akhir, program akan menampilkan visualisasi graf rute kereta api. Sehingga pengguna dapat melihat struktur graf posisi dari stasiun awal dan tujuan akhir.

E. Mencari jalur terpendek, total jarak yang ditempuh dan visualisasinya

```

# Menggunakan fungsi find shortest path untuk mencari jalur yang ditempuh
path_traveled = directed(G, start_station, end_station)

# Menampilkan total jarak yang ditempuh
total_distance = calculate_total_distance(G, path_traveled)

# Menampilkan hasil
print("Jalur yang ditempuh dari (start_station) ke (end_station): (path_traveled)")
print("Total jarak yang ditempuh: (total_distance) km")

# Membuat subgraf hanya dengan edge yang terdapat pada jalur yang ditempuh
highlighted_edges = [(path_traveled[i], path_traveled[i + 1]) for i in range(len(path_traveled) - 1)]
H = G.edge_subgraph(highlighted_edges)

# Menampilkan graf dengan jalur yang ditempuh
pos_highlighted = nx.spring_layout(H)
nx.draw(H, pos_highlighted, with_labels=True, node_size=700, node_color='skyblue', font_size=10, font_color='black', arrows=True, font_weight='bold')
labels_highlighted = nx.get_edge_attributes(H, 'weight')
nx.draw_networkx_edge_labels(H, pos_highlighted, edge_labels=labels_highlighted)

plt.show()

if __name__ == "__main__":
    main()

```

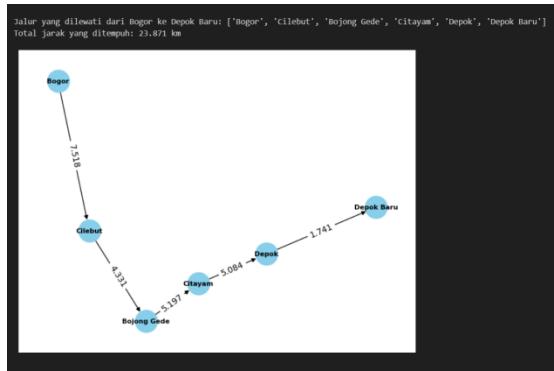
Gambar 3.7

Pada bagian program akan mencari jalur yang dilewati menggunakan fungsi *directed*, lalu program akan menghitung total dari jarak yang ditempuh dan menampilkannya hasilnya. Terakhir, perogram akan membuat subgraf dengan mempertahankan edge yang terdapat pada jalur yang dilewati, lalu menampilkannya dengan graf tersebut.

F. Output



Gambar 3.8



Gambar 3.9

IV. Kesimpulan

Dari hasil dan pembahasan dapat disimpulkan bahwa program ini menggunakan pustaka NetworkX dan Matplotlib untuk memvisualisasikan graf. Program akan mengarahkan pengguna untuk memasukkan stasiun awal dan tujuan akhir, lalu nantinya program akan melakukan pencarian jalur terpendek antara stasiun di Jabodetabek. Program dapat memvisualisasikan graf rute kereta api sehingga pengguna dapat melihat struktur jalur dan posisi stasiun dalam graf, selain itu program juga memberikan informasi jarak total yang ditempuh antar stasiun.

Secara keseluruhan, program ini dapat digunakan sebagai alat interaktif untuk membantu mengetahui rute perjalanan kereta api di Jabodetabek dengan memberikan informasi visual dan numerik mengenai jalur terpendek dan total jarak yang ditempuh antar dua stasiun.

Daftar Pustaka

[1] S . Wenas, "Pemodelan dan penjadwalan sistem jaringan KRL Commuter Line Jabodetabek dengan menggunakan Aljabar Max-Plus," p. 1, 2023

[2] L. Oktavia, H. Wulandjani and D. Riskarini, "ANALISIS PENGARUH FAKTOR-FAKTOR KUALITAS PELAYANAN DAN TEKNOLOGI INFORMASI TERHADAP KEPUASAN PELANGGAN JASA TRANSPORTASI PADA KRL COMMUTER LINE JABODETABEK (STUDI KASUS PADA MAHASISWI REGULER KHUSUS FEB UNIVERSITAS PANCASILA)," *Analisis Pengaruh Faktor-Faktor Kualitas Pelayanan Dan Teknologi Informasi*, vol. 2, p. 14, 2022.

[3] Iis, Sumiati. "Social Network Analysis Community". 2014

[4] RevoU. "Mengenal Mengenal Matplotlib dalam Python untuk Visualisasi Data". 2023.

[5] Tou, Nurhaeka. "Analisis Hubungan Kausal dari Faktor-Faktor yang Memengaruhi Kemandirian Lansia dalam Melakukan Activities of Daily Livings (ADLs)". 2020.